实验一、ROS 基础

一、 实验目的

1. 了解和掌握 ROS 操作系统的基础。
2. 学习如何在 ROS 操作系统中创建工作空间。
3. 学习如何实现 ROS 通信编程。
4. 学习如何在 ROS 操作系统中实现分布式通信。
5. 了解和掌握 ROS 中的关键组件。

二、 实验原理及说明

1.ROS 操作系统

ROS（Robot Operating System）是一个适用于机器人的开源的元操作系统。它提供了操作系统应有的服务，包括硬件抽象，底层设备控制，常用函数的实现，进程间消息传递，以及包管理。它也提供用于获取、编译、编写、和跨计算机运行代码所需的工具和库函数。

ROS 总体设计可以概括为：

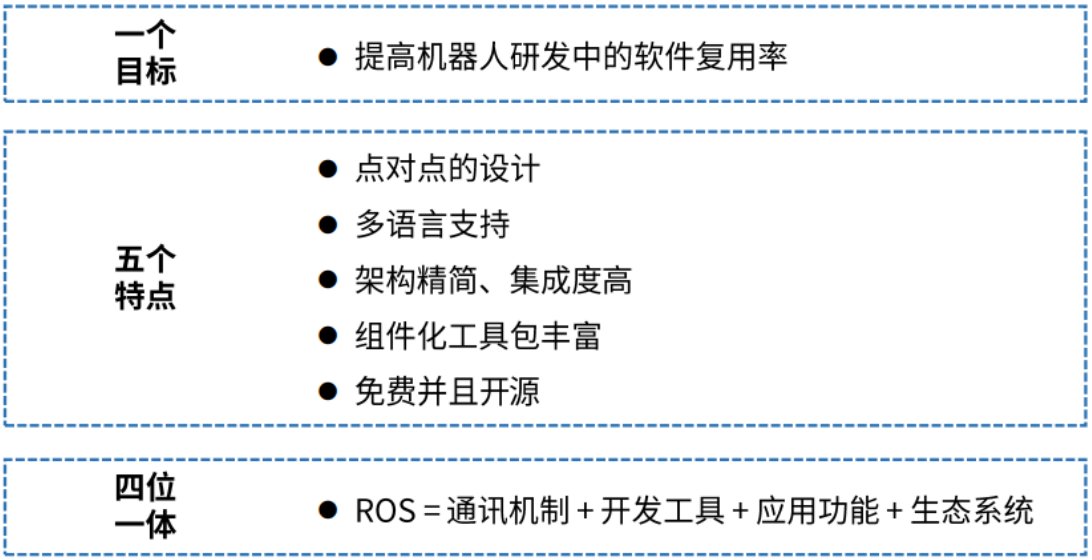

图 2.1 ROS 总体设计

2.工作空间

工作空间(workspace）是一个存放工程开发相关文件的文件夹。

（1）：src:代码空间(Source Space)
（2）：build:编译空间(Build Space)
（3）：devel:开发空间(Development Space)
（4）：install:安装空间(Install Space)

```
workspace_folder/          -- WORKSPACE
  src/                     -- SOURCE SPACE
    CMakeLists.txt         -- The 'toplevel' CMake file
    package_1/
      CMakeLists.txt
      package.xml
      ...
    package_n/
      CMakeLists.txt
      package.xml
      ...
  build/                   -- BUILD SPACE
    CATKIN_IGNORE          -- Keeps catkin from walking this directory
  devel/                   -- DEVELOPMENT SPACE (set by CATKIN_DEVEL_PREFIX)
    bin/
    etc/
    include/
    lib/
    share/
    .catkin
    env.bash
    setup.bash
    setup.sh
    ...
  install/                 -- INSTALL SPACE (set by CMAKE_INSTALL_PREFIX)
    bin/
    etc/
    include/
    lib/
    share/
    .catkin
    env.bash
    setup.bash
    setup.sh
    ...
```

图 2.2 catkin 编译系统下的工作空间结构

3. ROS 通信编程

3.1 话题编程

（1）话题编程流程

- 创建发布者
- 创建订阅者
- 添加编译选项
- 运行可执行程序

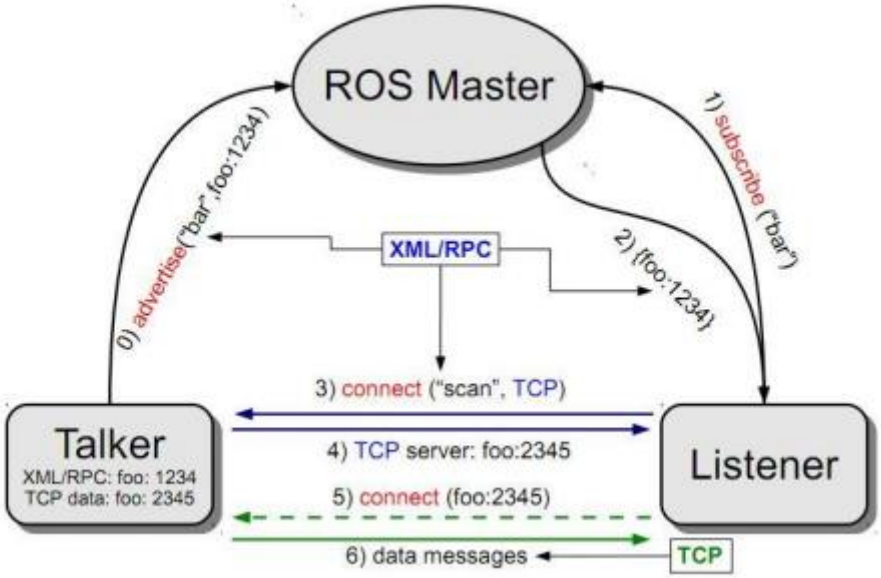

图 2.3 话题通信模型

3.2 服务编程

（1）服务编程流程

- 创建服务器
- 创建客户端
- 添加编译选项
- 运行可执行程序

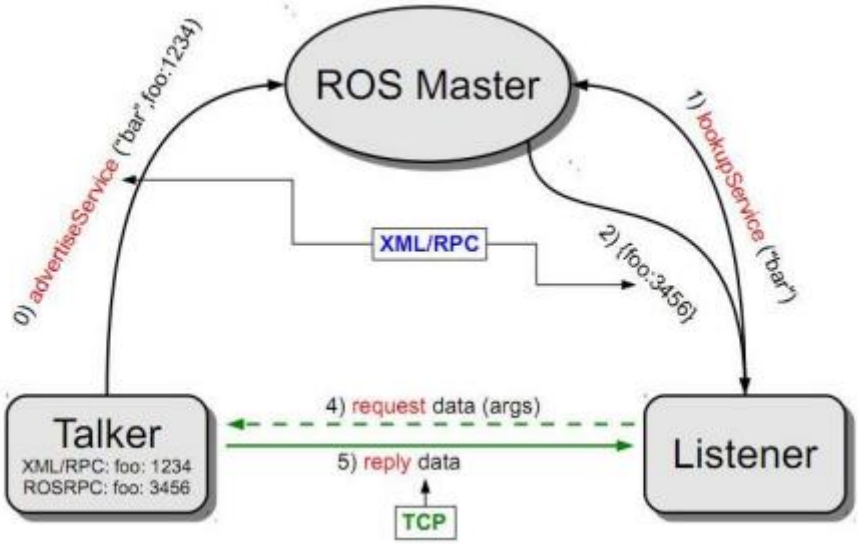

图 2.4 服务通信模型

3.3 动作编程

（1）动作的介绍

- 一种问答通信机制；
- 带有连续反馈；
- 可以在任务过程中止运行；
- 基于 ROS 的消息机制实现。

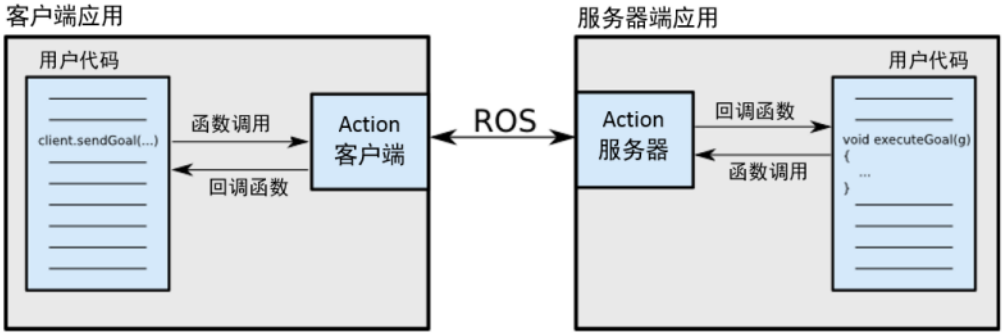

图 2.5 动作

（2）Action 的接口

- goal:发布任务目标；
- cancel:请求取消任务；
- status:通知客户端当前的状态；
- feedback:周期反馈任务运行的监控数据；
- result:向客户端发送任务的执行结果，只发布一次。

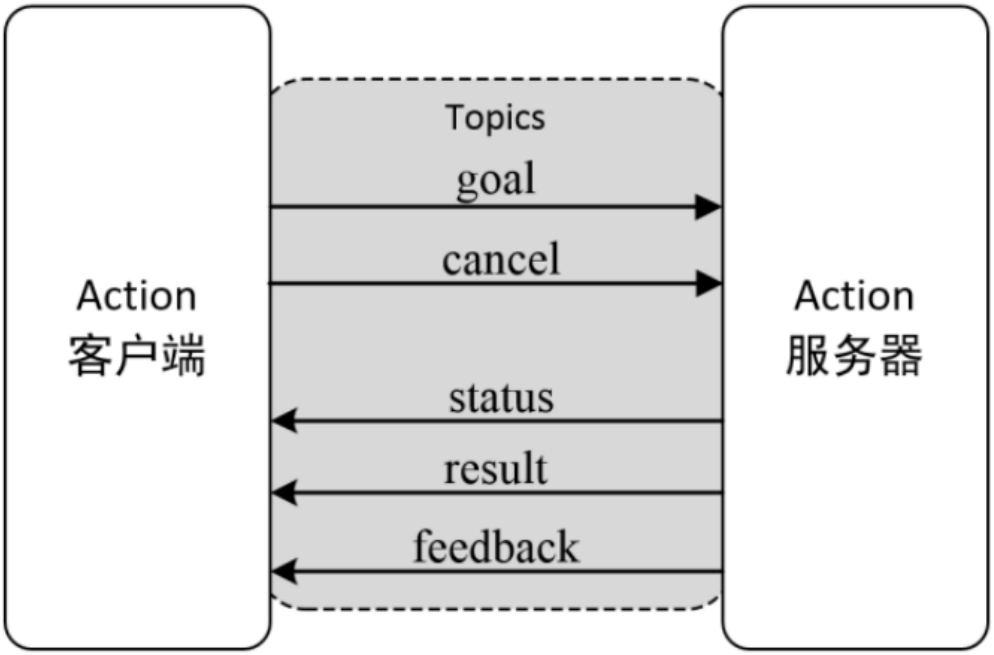

图 2.6 Action

4.分布式通信

ROS 是一种分布式软件框架，节点之间通过松耦合的方式进行组合。

5.ROS 中的关键组件

5.1 Launch 文件

Launch 文件:通过 XML 文件实现多节点的配置和启动(可自动启动 ROS Master)

```xml
<launch>
  <!-- local machine already has a definition by default.
       This tag overrides the default definition with
       specific ROS_ROOT and ROS_PACKAGE_PATH values -->
  <machine name="local_alt" address="localhost" default="true" ros-root="/u/user/ros/ros/" ros-package-path="/u/user/ros/ros-pkg" />
  <!-- a basic listener node -->
  <node name="listener-1" pkg="rospy_tutorials" type="listener" />
  <!-- pass args to the listener node -->
  <node name="listener-2" pkg="rospy_tutorials" type="listener" args="-foo arg2" />
  <!-- a respawn-able listener node -->
  <node name="listener-3" pkg="rospy_tutorials" type="listener" respawn="true" />
  <!-- start listener node in the 'wg1' namespace -->
  <node ns="wg1" name="listener-wg1" pkg="rospy_tutorials" type="listener" respawn="true" />
  <!-- start a group of nodes in the 'wg2' namespace -->
  <group ns="wg2">
    <!-- remap applies to all future statements in this scope. -->
    <remap from="chatter" to="hello"/>
    <node pkg="rospy_tutorials" type="listener" name="listener" args="--test" respawn="true" />
    <node pkg="rospy_tutorials" type="talker" name="talker">
      <!-- set a private parameter for the node -->
      <param name="talker_1_param" value="a value" />
      <!-- nodes can have their own remap args -->
      <remap from="chatter" to="hello-1"/>
      <!-- you can set environment variables for a node -->
      <env name="ENV_EXAMPLE" value="some value" />
    </node>
  </group>
</launch>
```

图 2.7 Launch 文件示例

（1）参数设置

<param> / <rosparam>

设置ROS系统运行中的参数，存储在参数服务器中。

<param name="output_frame" value="odom"/>

- name：参数名
- value：参数值

加载参数文件中的多个参数：

<rosparam file="params.yaml" command="load" ns= “params" />

<arg>

launch文件内部的局部变量，仅限于launch文件使用

<arg name="arg-name" default="arg-value" />

- name：参数名
- value：参数值

调用：

<param name="foo" value="$(arg arg-name)" />

<node name="node" pkg="package" type="type " args="$(arg arg-name)" />

图 2.8 参数设置

（2）重映射

<remap >

重映射ROS计算图资源的命名。

<remap from="/turtlebot/cmd_vel" to="/cmd_vel"/>

- from：原命名
- to：映射之后的命名

图 2.9 重映射

（3）嵌套

<include>	包含其他launch文件，类似C语言中的头文件包含。 <include file="$(dirname)/other.launch" /> • file：包含的其他launch文件路径

图 2.10 嵌套

5.2 QT 工具箱

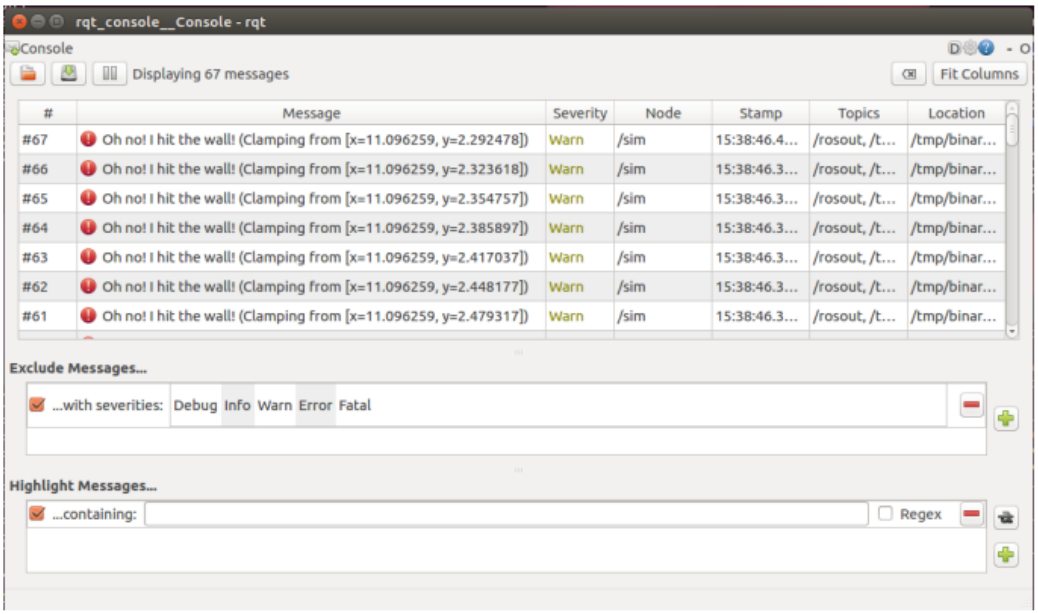

图 2.11 日志输出工具——rqt_console

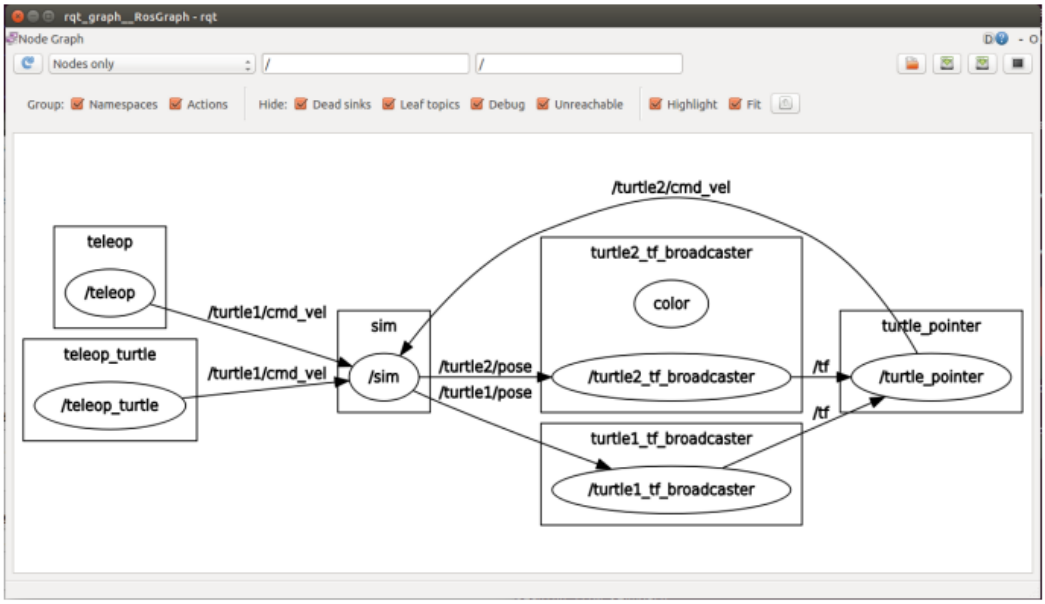

图 2.12 计算图可视化工具——rqt_graph

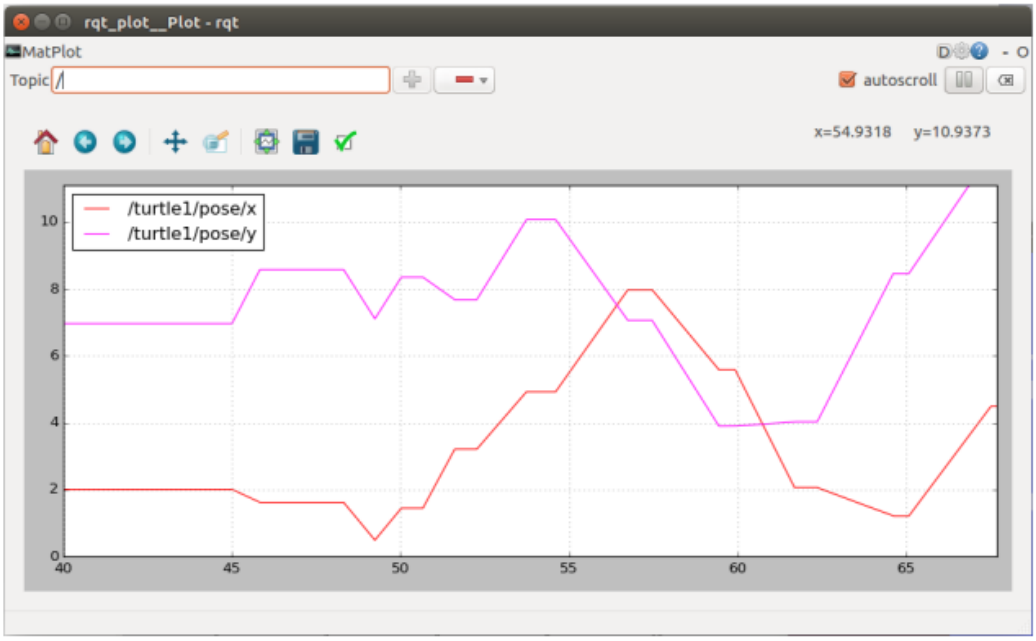

图 2.13 数据绘图工具——rqt_plot

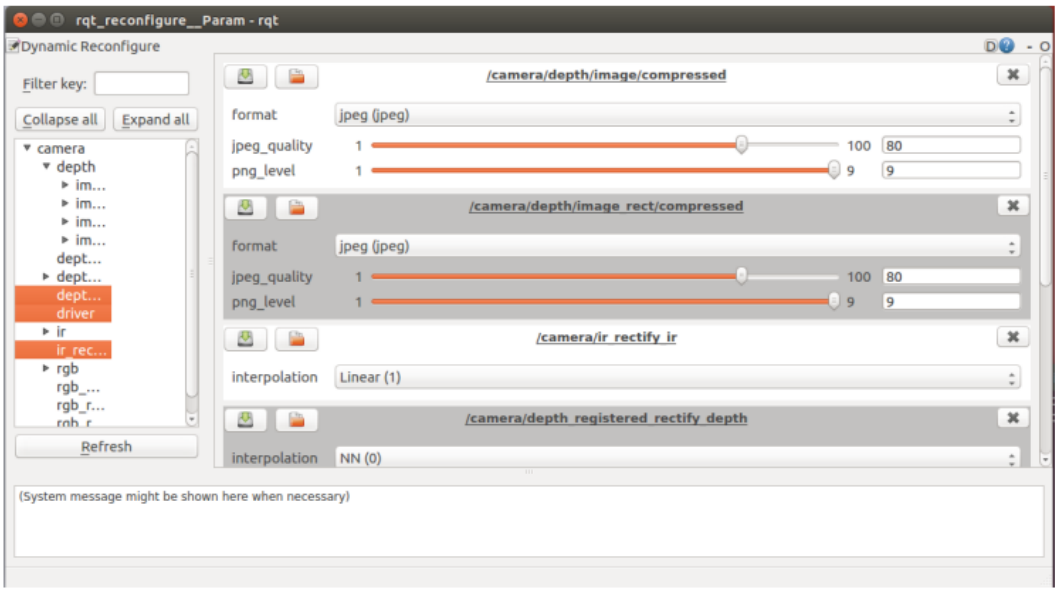

图 2.14 参数动态配置工具——rqt_reconfigure

5.3 Rviz 可视化平台

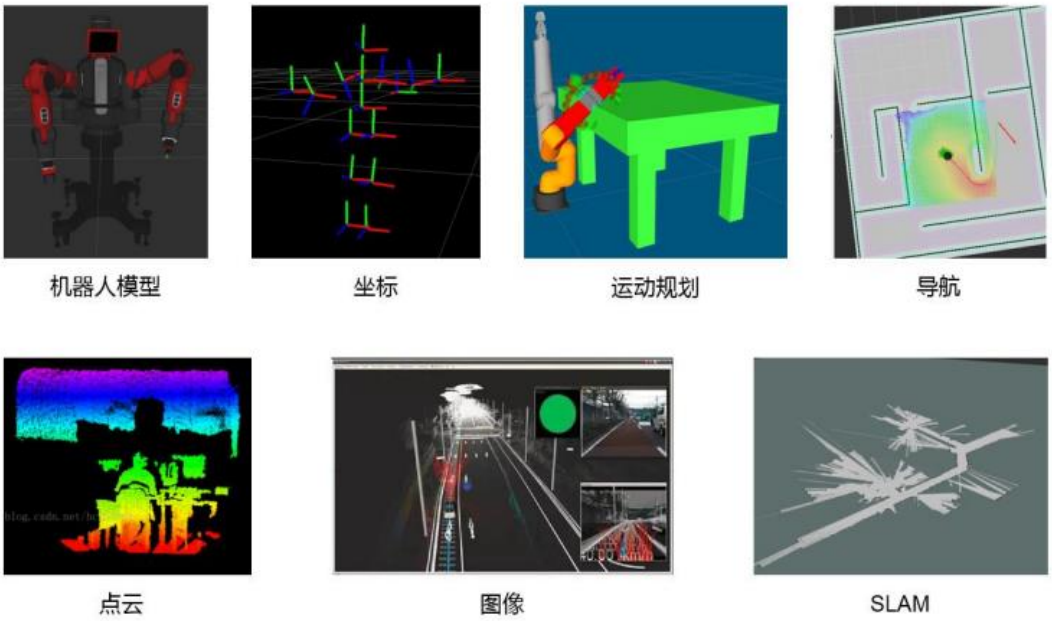

图 2.15 rviz 示例

Rviz 是一款三维可视化工具，可以很好的兼容基于 ROS 软件框架的机器人平台。

在 rviz 中，可以使用可扩展标记语言 XML 对机器人、周围物体等任何实物进行尺寸、质量、位置、材质、关节等属性的描述，并且在界面中呈现出来。同时，rviz 还可以通过图形化的方式，实时显示机器人传感器的信息、机器人的运动状态、周围环境的变化等信息。总而言之，rviz 通过机器人模型参数、机器人发布的传感信息等数据，为用户进行所有可监测信息的图形化显示。用户和开发者也可以在 rviz 的控制界面下，通过按钮、滑动条、数值等方式，控制机器人的行为。

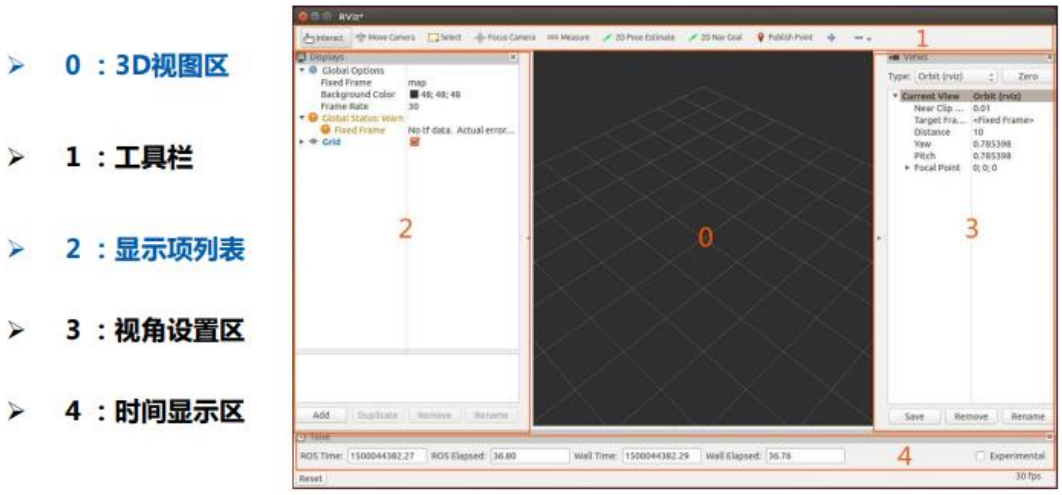

图 2.16 rviz 平台

5.4 Gazebo 物理仿真环境

Gazebo 是一款功能强大的三维物理仿真平台。

（1）具备强大的物理引擎

（2）高质量的图形渲染

（3）方便的编程与图形接口
（4）开源免费
其典型应用场景包括：
（1）测试机器人算法
（2）机器人的设计
（3）现实情景下的回溯测试

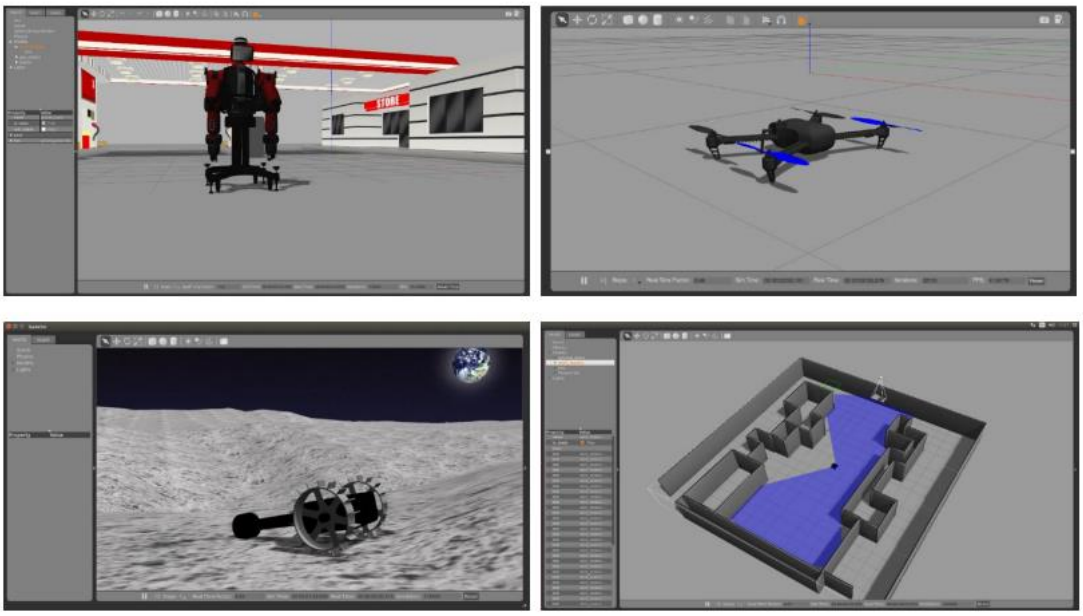

图 2.17 Gazebo 示例

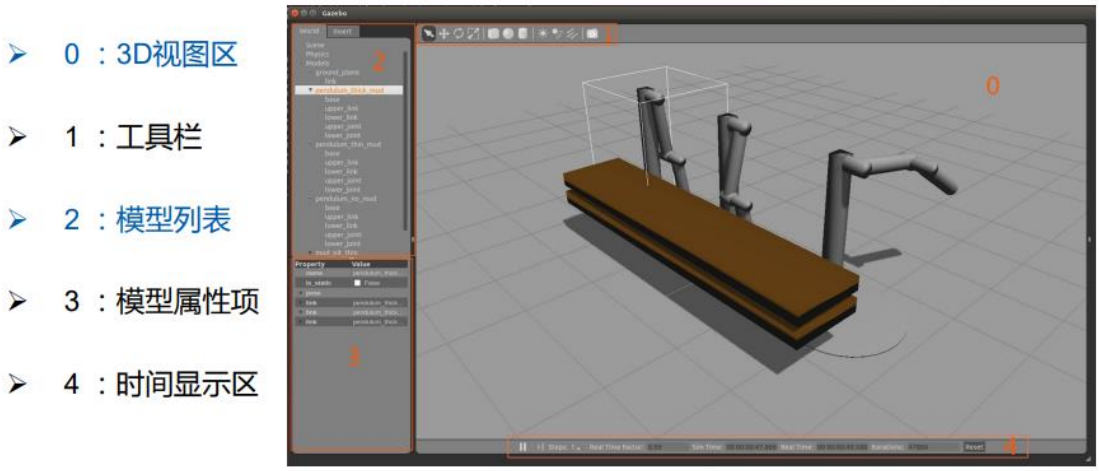

图 2.18 Gazebo 工程

三、 实验内容及步骤

1. 工作空间
1. 1 创建工作空间

创建工作空间

```
$ mkdir -p ~/catkin_ws/src
$ cd ~/catkin_ws/src
$ catkin_init_workspace
```

编译工作空间

```
$ cd ~/catkin_ws/
$ catkin_make
```

设置环境变量

```
$ source devel/setup.bash
```

检查环境变量

```
$ echo $ROS_PACKAGE_PATH
```

```
→  ~ echo $ROS_PACKAGE_PATH
/home/hcx/catkin_ws/src:/opt/ros/indigo/share:/opt/ros/indigo/stacks
```

图 3.1 创建工作空间

1.2 创建功能包

$ catkin_create_pkg <package_name> [depend1] [depend2] [depend3]

创建功能包

```
$ cd ~/catkin_ws/src
$ catkin_create_pkg learning_communication std_msgs rospy roscpp
```

编译功能包

```
$ cd ~/catkin_ws
$ catkin_make
$ source ~/catkin_ws/devel/setup.bash
```

同一个工作空间下，不允许存在同名功能包
不同工作空间下，允许存在同名功能包

图 3.2 创建功能包

1.3 工作空间的覆盖

ROS 工作空间的 Overlaying 机制，即工作空间的覆盖。

```
→  ~ env | grep ros
PATH=/home/hcx/catkin_ws/devel/bin:/opt/ros/kinetic/bin:/home/hcx/bin:/home/hcx/.local/bin:
/opt/jdk1.8.0_144/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/gam
es:/usr/local/games:/snap/bin
CMAKE_PREFIX_PATH=/home/hcx/catkin_ws/devel:/opt/ros/kinetic
LD_LIBRARY_PATH=/home/hcx/catkin_ws/devel/lib:/opt/ros/kinetic/lib:/opt/ros/kinetic/lib/x86
_64-linux-gnu
PKG_CONFIG_PATH=/home/hcx/catkin_ws/devel/lib/pkgconfig:/opt/ros/kinetic/lib/pkgconfig:/opt
/ros/kinetic/lib/x86_64-linux-gnu/pkgconfig
PYTHONPATH=/home/hcx/catkin_ws/devel/lib/python2.7/dist-packages:/opt/ros/kinetic/lib/pytho
n2.7/dist-packages
ROS_PACKAGE_PATH=/home/hcx/catkin_ws/src:/opt/ros/kinetic/share
ROS_ETC_DIR=/opt/ros/kinetic/etc/ros
ROS_ROOT=/opt/ros/kinetic/share/ros
TURTLEBOT_GAZEBO_MAP_FILE=/opt/ros/kinetic/share/turtlebot_gazebo/maps/playground.yaml
TURTLEBOT_GAZEBO_WORLD_FILE=/opt/ros/kinetic/share/turtlebot_gazebo/worlds/playground.world
TURTLEBOT_MAP_FILE=/opt/ros/kinetic/share/turtlebot_navigation/maps/willow-2010-02-18-0.10.
yaml
TURTLEBOT_STAGE_MAP_FILE=/opt/ros/kinetic/share/turtlebot_stage/maps/maze.yaml
TURTLEBOT_STAGE_WORLD_FILE=/opt/ros/kinetic/share/turtlebot_stage/maps/stage/maze.world
TURTLEBOT_STDR_MAP_FILE=/opt/ros/kinetic/share/turtlebot_stdr/maps/sparse_obstacles.yaml
```

图 3.3 查看 ROS 相关的环境变量

- 工作空间的路径依次在 ROS_PACKAGE_PATH 环境变量中记录。
- 新设置的路径在 ROS_PACKAGE_PATH 中会自动放置在最前端。
- 运行时，ROS 会优先查找最前端的工作空间中是否存在指定的功能包。
- 如果不存在，就顺序向后查找其他工作空间。

```
→  ~ rospack find roscpp_tutorials
/opt/ros/kinetic/share/roscpp_tutorials
```

图 3.4 系统路径下的功能包

```
→  catkin_ws rospack find roscpp_tutorials
/home/hcx/catkin_ws/src/ros_tutorials/roscpp_tutorials
```

图 3.5 工作空间下的功能包

2. ROS 通信编程

2.1 话题编程

（1）如何实现一个发布者

- 初始化 ROS 节点；
- 向 ROS Master 注册节点信息，包括发布的话题名和话题中的消息类型；
- 按照一定频率循环发布消息。

```
#include <sstream>
#include "ros/ros.h"
#include "std_msgs/String.h"

int main(int argc, char **argv)
{
    // ROS节点初始化
    ros::init(argc, argv, "talker");

    // 创建节点句柄
    ros::NodeHandle n;

    // 创建一个Publisher，发布名为chatter的topic，消息类型为std_msgs::String
    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);

    // 设置循环的频率
    ros::Rate loop_rate(10);

    int count = 0;
    while (ros::ok())
    {
        // 初始化std_msgs::String类型的消息
        std_msgs::String msg;
        std::stringstream ss;
        ss << "hello world " << count;
        msg.data = ss.str();

        // 发布消息
        ROS_INFO("%s", msg.data.c_str());
        chatter_pub.publish(msg);

        // 循环等待回调函数
        ros::spinOnce();

        // 按照循环频率延时
        loop_rate.sleep();
        ++count;
    }

    return 0;
}
```

talker.cpp

图 3.6 发布者 C 文件

（2）如何实现一个订阅者

- 初始化 ROS 节点；
- 订阅需要的话题；
- 循环等待话题消息，接收到消息后进入回调函数；
- 在回调函数中完成消息处理。

```
#include "ros/ros.h"
#include "std_msgs/String.h"

// 接收到订阅的消息后，会进入消息回调函数
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
    // 将接收到的消息打印出来
    ROS_INFO("I heard: [%s]", msg->data.c_str());
}

int main(int argc, char **argv)
{
    // 初始化ROS节点
    ros::init(argc, argv, "listener");

    // 创建节点句柄
    ros::NodeHandle n;

    // 创建一个Subscriber，订阅名为chatter的topic，注册回调函数chatterCallback
    ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);

    // 循环等待回调函数
    ros::spin();

    return 0;
}
```

listener.cpp

图 3.7 订阅者 C 文件

（3）编译代码

- 设置需要编译的代码和生成的可执行文件；
- 设置链接库；
- 设置依赖。

```
add_executable(talker src/talker.cpp)
target_link_libraries(talker ${catkin_LIBRARIES})
# add_dependencies(talker ${PROJECT_NAME}_generate_messages_cpp)

add_executable(listener src/listener.cpp)
target_link_libraries(listener ${catkin_LIBRARIES})
# add_dependencies(talker ${PROJECT_NAME}_generate_messages_cpp)
```

图 3.8 CMakeLists.txt

（4）运行可执行文件

```
➜  ~ rosrun learning_communication talker
[ INFO] [1507648965.972490912]: hello world 0
[ INFO] [1507648966.072604444]: hello world 1
[ INFO] [1507648966.172600924]: hello world 2
[ INFO] [1507648966.272562923]: hello world 3
[ INFO] [1507648966.372559424]: hello world 4
[ INFO] [1507648966.472688746]: hello world 5
[ INFO] [1507648966.572691987]: hello world 6
[ INFO] [1507648966.672685917]: hello world 7
[ INFO] [1507648966.772684550]: hello world 8
[ INFO] [1507648966.872679978]: hello world 9
[ INFO] [1507648966.972579883]: hello world 10
[ INFO] [1507648967.072628016]: hello world 11
```

rosrun learning_communication talker

```
➜  ~ rosrun learning_communication listener
[ INFO] [1507649032.961619694]: I heard: [hello world 56]
[ INFO] [1507649033.061319417]: I heard: [hello world 57]
[ INFO] [1507649033.161406093]: I heard: [hello world 58]
[ INFO] [1507649033.260928830]: I heard: [hello world 59]
[ INFO] [1507649033.361426764]: I heard: [hello world 60]
[ INFO] [1507649033.461283961]: I heard: [hello world 61]
[ INFO] [1507649033.561306202]: I heard: [hello world 62]
[ INFO] [1507649033.661385283]: I heard: [hello world 63]
[ INFO] [1507649033.761384508]: I heard: [hello world 64]
[ INFO] [1507649033.861318444]: I heard: [hello world 65]
[ INFO] [1507649033.961297490]: I heard: [hello world 66]
[ INFO] [1507649034.061417031]: I heard: [hello world 67]
[ INFO] [1507649034.161411960]: I heard: [hello world 68]
```

rosrun learning_communication listener

图 3.9 运行可执行文件

2.2 服务编程

（1）如何实现一个服务器

- 初始化 ROS 节点；
- 创建 Server 实例；
- 循环等待服务请求，进入回调函数；
- 在回调函数中完成服务功能的处理，并反馈应答数据。

```
#include "ros/ros.h"
#include "learning_communication/AddTwoInts.h"

// service回调函数，输入参数req，输出参数res
bool add(learning_communication::AddTwoInts::Request  &req,
         learning_communication::AddTwoInts::Response &res)
{
    // 将输入参数中的请求数据相加，结果放到应答变量中
    res.sum = req.a + req.b;
    ROS_INFO("request: x=%ld, y=%ld", (long int)req.a, (long int)req.b);
    ROS_INFO("sending back response: [%ld]", (long int)res.sum);

    return true;
}

int main(int argc, char **argv)
{
    // ROS节点初始化
    ros::init(argc, argv, "add_two_ints_server");

    // 创建节点句柄
    ros::NodeHandle n;

    // 创建一个名为add_two_ints的server，注册回调函数add()
    ros::ServiceServer service = n.advertiseService("add_two_ints", add);

    // 循环等待回调函数
    ROS_INFO("Ready to add two ints.");
    ros::spin();

    return 0;
}
```

server.cpp

图 3.10 服务器 C 文件

（2）如何实现一个客户端

- 初始化 ROS 节点；
- 创建一个 Client 实例；
- 发布服务请求数据；
- 等待 Server 处理之后的应答结果。

```
#include <cstdlib>
#include "ros/ros.h"
#include "learning_communication/AddTwoInts.h"

int main(int argc, char **argv)
{
    // ROS节点初始化
    ros::init(argc, argv, "add_two_ints_client");

    // 从终端命令行获取两个加数
    if (argc != 3)
    {
        ROS_INFO("usage: add_two_ints_client X Y");
        return 1;
    }

    // 创建节点句柄
    ros::NodeHandle n;

    // 创建一个client，请求add_two_int service
    // service消息类型是learning_communication::AddTwoInts
    ros::ServiceClient client = n.serviceClient<learning_communication::AddTwoInts>("add_two_ints");

    // 创建learning_communication::AddTwoInts类型的service消息
    learning_communication::AddTwoInts srv;
    srv.request.a = atoll(argv[1]);
    srv.request.b = atoll(argv[2]);

    // 发布service请求，等待加法运算的应答结果
    if (client.call(srv))
    {
        ROS_INFO("Sum: %ld", (long int)srv.response.sum);
    }
    else
    {
        ROS_ERROR("Failed to call service add_two_ints");
        return 1;
    }

    return 0;
}
```

client.cpp

图 3.11 客户端 C 文件

（3）编译代码

- 设置需要编译的代码和生成的可执行文件；
- 设置链接库；
- 设置依赖。

```
add_executable(server src/server.cpp)
target_link_libraries(server ${catkin_LIBRARIES})
add_dependencies(server ${PROJECT_NAME}_gencpp)

add_executable(client src/client.cpp)
target_link_libraries(client ${catkin_LIBRARIES})
add_dependencies(client ${PROJECT_NAME}_gencpp)
```

图 3.12 CMakeLists.txt

（4）运行可执行文件

```
➜  ~ rosrun learning_communication server
[ INFO] [1507649760.914978873]: Ready to add two ints.
```

Server节点启动后的日志信息

```
➜  ~ rosrun learning_communication client 3 5
[ INFO] [1507649815.838663270]: Sum: 8
```

Client启动后发布服务请求，并成功接收到反馈结果

```
➜  ~ rosrun learning_communication server
[ INFO] [1507649760.914978873]: Ready to add two ints.

[ INFO] [1507649815.838470408]: request: x=3, y=5
[ INFO] [1507649815.838508903]: sending back response: [8]
```

Server接收到服务调用后完成加法求解，并将结果反馈给Client

图 3.13 运行可执行文件

2.3 动作编程

（1）如何实现一个动作服务器

- 初始化 ROS 节点；
- 创建动作服务器实例；
- 启动服务器，等待动作请求；
- 在回调函数中完成动作服务功能的处理，并反馈进度信息；
- 动作完成，发送结束信息。

```
#include <ros/ros.h>
#include <actionlib/server/simple_action_server.h>
#include "learning_communication/DoDishesAction.h"

typedef actionlib::SimpleActionServer<learning_communication::DoDishesAction> Server;

// 收到action的goal后调用该回调函数
void execute(const learning_communication::DoDishesGoalConstPtr& goal, Server* as)
{
    ros::Rate r(1);
    learning_communication::DoDishesFeedback feedback;

    ROS_INFO("Dishwasher %d is working.", goal->dishwasher_id);

    // 假设洗盘子的进度，并且按照1hz的频率发布进度feedback
    for(int i=1; i<=10; i++)
    {
        feedback.percent_complete = i * 10;
        as->publishFeedback(feedback);
        r.sleep();
    }

    // 当action完成后，向客户端返回结果
    ROS_INFO("Dishwasher %d finish working.", goal->dishwasher_id);
    as->setSucceeded();
}

int main(int argc, char** argv)
{
    ros::init(argc, argv, "do_dishes_server");
    ros::NodeHandle n;

    // 定义一个服务器
    Server server(n, "do_dishes", boost::bind(&execute, _1, &server), false);

    // 服务器开始运行
    server.start();

    ros::spin();

    return 0;
}
```

DoDishes_server.cpp

图 3.14 动作服务器 C 文件

（2）如何实现一个动作客户端

- 初始化 ROS 节点；
- 创建动作客户端实例；
- 连接动作服务端；
- 发送动作目标；
- 根据不同类型的服务端反馈处理回调函数。

```cpp
#include <actionlib/client/simple_action_client.h>
#include "learning_communication/DoDishesAction.h"

typedef actionlib::SimpleActionClient<learning_communication::DoDishesAction> Client;

// 当action完成后会调用该回调函数一次
void doneCb(const actionlib::SimpleClientGoalState& state,
        const learning_communication::DoDishesResultConstPtr& result)
{
    ROS_INFO("Yay! The dishes are now clean");
    ros::shutdown();
}

// 当action激活后会调用该回调函数一次
void activeCb()
{
    ROS_INFO("Goal just went active");
}

// 收到feedback后调用该回调函数
void feedbackCb(const learning_communication::DoDishesFeedbackConstPtr& feedback)
{
    ROS_INFO(" percent_complete : %f ", feedback->percent_complete);
}

int main(int argc, char** argv)
{
    ros::init(argc, argv, "do_dishes_client");

    // 定义一个客户端
    Client client("do_dishes", true);

    // 等待服务器端
    ROS_INFO("Waiting for action server to start.");
    client.waitForServer();
    ROS_INFO("Action server started, sending goal.");

    // 创建一个action的goal
    learning_communication::DoDishesGoal goal;
    goal.dishwasher_id = 1;

    // 发送action的goal给服务器端，并且设置回调函数
    client.sendGoal(goal,  &doneCb, &activeCb, &feedbackCb);

    ros::spin();

    return 0;
}
```

DoDishes_client.cpp

图 3.15 动作客户端 C 文件

（3）编译代码

- 设置需要编译的代码和生成的可执行文件；
- 设置链接库；
- 设置依赖。

```
add_executable(DoDishes_client src/DoDishes_client.cpp)
target_link_libraries( DoDishes_client ${catkin_LIBRARIES})
add_dependencies(DoDishes_client ${${PROJECT_NAME}_EXPORTED_TARGETS})

add_executable(DoDishes_server src/DoDishes_server.cpp)
target_link_libraries( DoDishes_server ${catkin_LIBRARIES})
add_dependencies(DoDishes_server ${${PROJECT_NAME}_EXPORTED_TARGETS})
```

图 3.16 CMakeLists.txt

（4）运行可执行文件

```
hcx@hcx-pc: ~
➜  ~ rosrun learning_communication DoDishes_client
[ INFO] [1522297575.233018062]: Waiting for action server to start.
[ INFO] [1522297585.563888038]: Action server started, sending goal.
[ INFO] [1522297585.565124594]: Goal just went active
[ INFO] [1522297585.565291254]:  percent_complete : 10.000000
[ INFO] [1522297586.565387224]:  percent_complete : 20.000000
[ INFO] [1522297587.565365764]:  percent_complete : 30.000000
[ INFO] [1522297588.565402309]:  percent_complete : 40.000000
[ INFO] [1522297589.565335020]:  percent_complete : 50.000000
[ INFO] [1522297590.565434414]:  percent_complete : 60.000000
[ INFO] [1522297591.565405379]:  percent_complete : 70.000000
[ INFO] [1522297592.565433372]:  percent_complete : 80.000000
[ INFO] [1522297593.565421092]:  percent_complete : 90.000000
[ INFO] [1522297594.565399486]:  percent_complete : 100.000000
[ INFO] [1522297595.565578664]: Yay! The dishes are now clean
➜  ~
rosrun learning_communication DoDishes_server
➜  ~ rosrun learning_communication DoDishes_server
[ INFO] [1522297585.564824906]: Dishwasher 1 is working.
[ INFO] [1522297595.564966474]: Dishwasher 1 finish working.
```

图 3.17 action 例程的运行效果

3. 分布式通信

3.1 分布式多机通信

（1）设置 IP 地址，确保底层链路的联通

```
wlp2s0    Link encap:Ethernet  HWaddr ac:2b:6e:5d:cc:85
          inet addr:192.168.31.198  Bcast:192.168.31.255  Mask:255.255.255.0
          inet6 addr: fe80::6751:3a89:703f:7e9c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:11169 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7686 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10038665 (10.0 MB)  TX bytes:1153112 (1.1 MB)

wlxc83a35b01533 Link encap:Ethernet  HWaddr c8:3a:35:b0:15:33
          inet addr:192.168.31.14  Bcast:192.168.31.255  Mask:255.255.255.0
          inet6 addr: fe80::2c61:6fea:37e4:69fe/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:287 errors:0 dropped:0 overruns:0 frame:0
          TX packets:68 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:97160 (97.1 KB)  TX bytes:11068 (11.0 KB)
```

图 3.18 两台计算机（hcx-pc、raspi2）的 IP 地址

```
hcx@hcx-pc:~$ ping raspi2
PING raspi2 (192.168.31.14) 56(84) bytes of data.
64 bytes from raspi2 (192.168.31.14): icmp_seq=1 ttl=64 time=2.24 ms
64 bytes from raspi2 (192.168.31.14): icmp_seq=2 ttl=64 time=2.37 ms
64 bytes from raspi2 (192.168.31.14): icmp_seq=3 ttl=64 time=2.25 ms
64 bytes from raspi2 (192.168.31.14): icmp_seq=4 ttl=64 time=2.20 ms
64 bytes from raspi2 (192.168.31.14): icmp_seq=5 ttl=64 time=2.14 ms
```

```
robot@raspi2:~$ ping hcx-pc
PING hcx-pc (192.168.31.198) 56(84) bytes of data.
64 bytes from hcx-pc (192.168.31.198): icmp_seq=1 ttl=64 time=9.68 ms
64 bytes from hcx-pc (192.168.31.198): icmp_seq=2 ttl=64 time=6.39 ms
64 bytes from hcx-pc (192.168.31.198): icmp_seq=3 ttl=64 time=1.76 ms
64 bytes from hcx-pc (192.168.31.198): icmp_seq=4 ttl=64 time=5.32 ms
64 bytes from hcx-pc (192.168.31.198): icmp_seq=5 ttl=64 time=7.05 ms
64 bytes from hcx-pc (192.168.31.198): icmp_seq=6 ttl=64 time=7.02 ms
```

图 3.19 在两台计算机上分别使用 ping 命令测试网络是否联通

（2）在从机端设置 ROS_MASTER_URI，让从机找到 ROS Master

```
$ export ROS_MASTER_URI=http://hcx-pc:11311   （当前终端有效）

或

$ echo "export ROS_MASTER_URI=http://hcx-pc:11311" >> ~/.bzshrc   （所有终端有效）
```

主机端启动ROS Master与海龟仿真节点

```
$ roscore
$ rosrun turtlesim turtlesim_node
```

从机端发布一个速度控制消息

```
$ rostopic pub -r 10 /turtle1/cmd_vel
  geometry_msgs/Twist "linear:
    x: 0.5
    y: 0.0
    z: 0.0
  angular:
    x: 0.0
    y: 0.0
    z: 0.5"
```

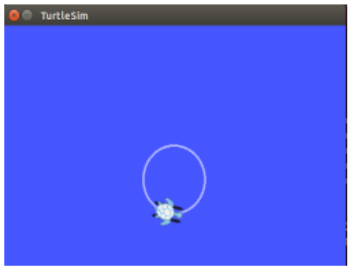

图 3.20 命令测试

四、 实验报告要求

（1）根据实验内容和步骤完成和记录实验过程和结果。
（2）掌握 ROS 的相关基础操作。
（3）掌握工作空间的建立以及 ROS 通信编程，实现分布式通讯。